

Shoot it down. Pick it up

1. Project Overview

For this project, the player is given a role as a trash collector of a town in the middle of a zombie apocalypse. The player's objective is to clean up the town of contaminated-trash and zombie remains as much as possible or else the town's people will get infected. If the infection rate is too high, it's game over.

2. Project Review

The relevant existing project in comparison would be Stardew Valley, an RPG game focusing on farming, foraging and killing monsters, the similarity is how the player interacts with its environment in the game. While Stardew Valley focus is on farming, this project is more focused on collecting and cleaning up "trash" and killing any zombie found around the map.

3. Programming Development

3.1 Game Concept

Describe the game in detail, including its mechanics and objectives. Highlight the key features or selling points of the game.

This game has 2 main Mechanics:

1. The player can pick up trash and zombie's remains within the map.
The game will progress into a different day when your character goes to bed. For each day that you did not clean up all the trash and zombie's remains, the infection rate of the people in the town will go up, once that level reaches 100%, it's game over.
2. There will be some zombie that is still alive wandering around that the player needs to **shoot*** down without it reaching and killing the player

as well. If the player gets attacked by zombies till their health reaches zero, it's game over as well.

The player character when idling will have their hand free and can pick up the trash, *to shoot, they need to press down a key for the player's character to pull out the gun, in this state, the player won't be able to pick up anything unless they press the trigger key again to put the gun back, returning to idle state. The game objective is to progress as many days as possible.

3.2 Object-Oriented Programming Implementation

Rough draft of OOP classes

Player : Control player input and movement and action like collecting trash within the player radius or shooting.

- Attributes: health: integer, damage: integer, image(for walking animation): sprite, state: "idle" or "waking", facing(which way the player is facing): "facing" or "back" etc.
- Method:
 - update sprite: update the character image depending on the direction the player is facing
 - move up, move down, move left, move right: update the player position.
 - sprinting: make the player move faster
 - get damage: decreasing the health depending on the damage the player got.
 - shoot: shooting in the the direction the player is facing
 - collecting: collect the trash within the player radius and calling trash collect from Level Progress
 - reset: reset all the stat

Zombie: Randomly spawn in each area within a random amount of 1-4. If the player is within its radius area, it will move toward the player and attack. When the zombie health reaches 0, it will disappear and drop trash.

- Attribute: health: integer, damage: integer, image: sprite, spawn position: tuple
- Method:
 - random spawn: spawn zombies at random locations within the current map the player enters.
 - attack: dealing damage to player when collided.

Trash: Randomly spawn trash in the map area the player is in within a random amount.

Game status: Check over the game progress with information from **Player** and check whether or not should the game let the player progress into the next day and draw the menu, game over and pause screen

- Attribute: trash collected, zombie killed (these 2 will be called up whenever the kill and collect method in **Player** got called up)
- Method:
 - calculate infection rate: calculate infection rate from the left over trash and zombies
 - new day: after the player going to bed in the game, it will check whether or not the infection rate has reached the limit yet, if it had, it'll call up the game over, if not, the game will progress into a new day
 - update status: if the game is not over, update the day and infection rate to the present ones
 - check infection: check if the infection rate has reached the limit yet, return value in bool, True if limit has been reached, otherwise return false,
 - calculate points: calculate point from trash collected and zombies killed
 - pauses screen: draw up the pause screen

Map Configure: Configure which map to load depending on the current map and the path the player chooses and check if the player collides into anything in the map.

- Attribute: images, current map/background
- Method:
 - map transition: change the map by checking the current map and the direction the player choose

3.3 Algorithms Involved

- Pathfinding Algorithm, grid based movement: For zombie movement
- Tile Based Placement: Choose placement for trash spawn points.
- Trigger-based spawning: Using Collision Trigger and Zone Transition, a random amount of zombies will spawn when the player steps into a new area.
- Collision Detection: Detect player's, trash's and zombie's hitbox.
- Inventory Management: Array or Dictionary to store trash items collected.
- Finite State Machine: To handle the model and animation of moving character like the player

4. Statistical Data (Prop Stats)

4.1 Data Features

1. Zombies killed: Number of zombie the player killed
2. Trash collected: Number of trash the player collected
3. Left over: Number of trash that was not collected and the zombie that was left alive.
4. Bullet fired: Number of bullet the player fired
5. Points: calculate points from amount of zombies and trash the player eliminated

Data	Amount
Zombies Killed, Trash collected,	These 3 data will be used and

Left over	calculated into percent to see how fast and efficient can the player eliminate the zombie and trash
Bullet fired	This data will be compared with the number of zombies the player killed to see if the player use their shots efficiently
Points	There will be 2 kinds of points data, one from trash collected and another from zombies killed. This data is used to see if the player is gaining points from killing zombies or collecting trash more. (Point gain from cleaning up zombie's remain will be count toward point of zombies killed rather than trash collected)

4.2 Data Recording Method

The data will be stored in CSV file format every 5 seconds. The data will be from the **Game status** class

4.3 Data Analysis Report

Data

	What can it be used for?	How to obtain 100 values of data	Which variable/class to collect the data from?	Display
Zombies killed	See the amount of zombies the player killed ever 5	As stated earlier, all 5 data will be collected every 5	All 5 data will be collected from class Game status	Bar graph

	seconds and will be used to calculate the elimination percentage	seconds, I'll manage the game time so it'll collect 100 values		
Trash collected	See the amount of trash the player collected every 5 seconds and will be used to calculate the elimination percentage	As stated earlier, all 5 data will be collected every 5 seconds, I'll manage the game time so it'll collect 100 values	All 5 data will be collected from class Game status	Bar graph
Left over	See the amount of left over trash and zombie every 5 seconds and will be used to calculate the elimination percentage	As stated earlier, all 5 data will be collected every 5 seconds, I'll manage the game time so it'll collect 100 values	All 5 data will be collected from class Game status	Bar graph
Elimination percentage	Calculate from 3 data, zombies killed, trash collected and left over. To see how fast and how efficient can the player eliminate the trash and	As stated earlier, all 5 data will be collected every 5 seconds, I'll manage the game time so it'll collect 100 values	All 5 data will be collected from class Game status	Bar graph

	zombie.			
Bullet fired	Use this data to compare with the number of zombies the player killed to see if the player use the bullet efficiently and how accurate is the player's aim	As stated earlier, all 5 data will be collected every 5 seconds, I'll manage the game time so it'll collect 100 values	All 5 data will be collected from class Game status	Line graph
Points	This data is to see how fast the player's score progress	As stated earlier, all 5 data will be collected every 5 seconds, I'll manage the game time so it'll collect 100 values	All 5 data will be collected from class Game status	Line graph

Graph

	Feature name	Graph Objective	Graph type	X-axis	Y-axis
Graph 1	Zombies killed, Trash	Visualizing the comparison	Bar graph	Time (0-5 sec, 5-10 sec, etc)	Amount of zombies the player

	collected, and Left over	of the 3 data			killed/trash collected and left over
Graph 2	Elimination percentage (Using 3 data, zombies killed, trash collected and left over)	See how efficient can the player clear the map every 5 second	Line graph	Time (0-5 sec, 5-10 sec, etc)	Percentage
Graph 3	Bullets vs Kills	Compare number of bullet fired with the number of zombies the player killed to show whether higher bullet usage correlates with higher kills, and highlights inefficiency (lots of bullets, few kills)	Scatter plot	Bullet fired	Zombies killed
Graph 4	Points vs Time	Show the points gained every 5 second to		Time (0-5 sec, 5-10 sec, etc)	Points

		spot bursts of scoring vs slower periods.			
Graph 5	Left over and Elimination Comparison	This graph is to visualize the comparison between Left over data and Elimination percentage data	Scatter plot	Left over	Elimination percentage
Graph 6	Points collection	Show proportion of points gained from zombies vs trash.	Pie chart		

Statistic Table

The data shown in the table will be shown from each game-day

Feature	Statistic
Points	Range, Mean, Median, Mode
Trash collected	Range, Mean, Median, Mode
Zombies killed	Range, Mean, Median, Mode

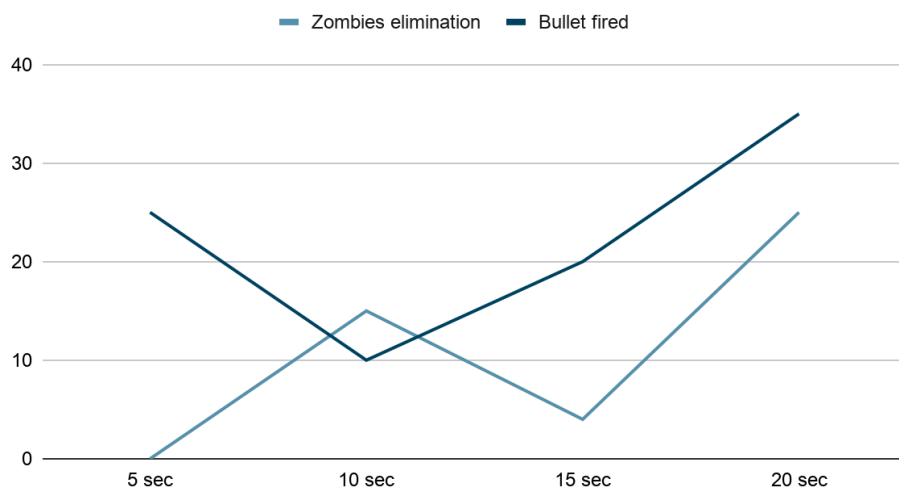
Zombies killed, Trash collected, Left over: These data will be shown in a bar graph. Draft example

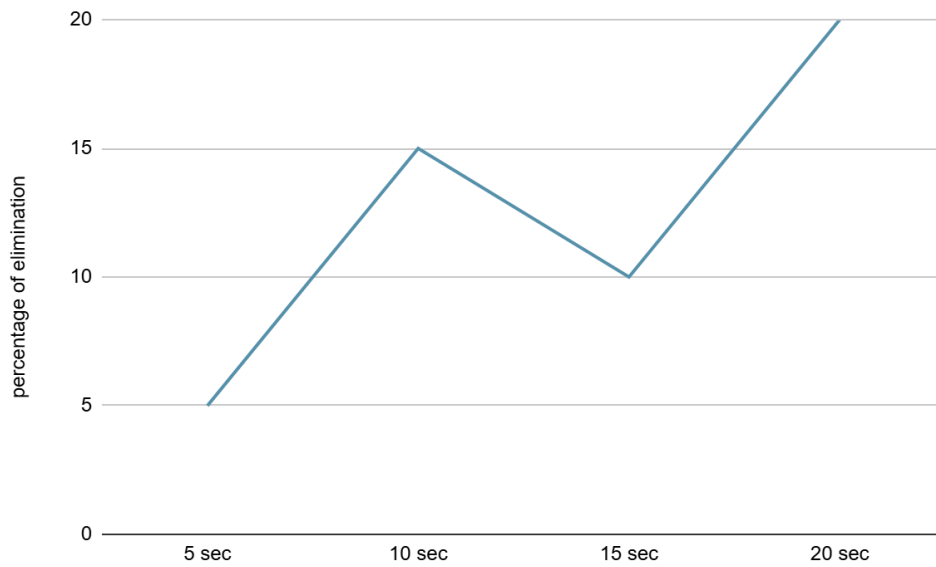
Eliminations and Left over



The bullets, the percentage of the efficiency of the player cleaning up the trash and zombie and the points data will be shown with a line graph. Draft example:

Points





5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Full proposal submission
3	10	Game mechanics
4	11	Game menu and interface
5	12	Data saving and visualization
6	13	Project revise
7	14	Submission week (Draft)

6. Document version

Version: 1.7

Date: 5 April 2026

